EEEE4138 Aerial Robotics Practical 2

Nonlinear Quadrotor Modelling and Validation

Build a nonlinear 12-state quadrotor model in Simulink and check its response against hand calculations.

Quick Start

Create both files in the same MATLAB folder.

1. Open your lab folder. Create Practical2_Sxx for your assigned ID and open it in MATLAB Current Folder.

2. Create the parameter script. Copy Section 2 into a new MATLAB script, set your assigned student_id and save it as practical2_parameters.m.

3. Build the Simulink model. Save a Blank Model as Practical2_Quadrotor.slx. Complete the MATLAB Function blocks in Sections 3 and 4 and connect the blocks and logs in Section 5.

4. Check the model. Save it and press Ctrl+D in Simulink. Fix any errors before running.

5. Run nominal hover. Enter both lines below in the MATLAB Command Window. Check that position and velocity stay close to zero.

```
practical2_parameters
out = sim('Practical2_Quadrotor');
```

6. Save your results. Follow Section 5 to save the hover run. For each new test, run the parameter script, apply the test changes, then simulate.

If MATLAB cannot find a file, check Current Folder and the exact filename.

Learning objectives

- Construct the force and moment mapping for the assigned plus or X configuration.
- Implement position kinematics, translational dynamics, attitude kinematics and rotational dynamics with consistent coordinates.
- Validate the model against physical predictions before adding a controller.

1 Preparation and working files

Create a folder Practical2_Sxx with a results subfolder. Create practical2_parameters.m and Practical2_Quadrotor.slx. Use MATLAB Function blocks for the two calculations described below; equivalent standard-block implementations are acceptable. Keep all parameter values in the script.

Conventions and assumptions

Use North–East–Down (NED) inertial coordinates and body axes x forward, y right, z down. Angles and angular rates are in radians and rad/s. R maps body vectors into NED. Rotor thrust is along negative body z; moments act about the centre of mass.

Assume a rigid aircraft with diagonal body inertia. Initially neglect drag, motor lag, rotor gyroscopic moments, wind and ground contact. Include drag only in its separate test. The point-payload convention changes mass without changing inertia.

2 Parameters and state definition

Copy values from the assigned parameter row. The benchmark below reproduces that allocation; use your assigned ID and cross-check the resulting values. Run the script before opening a simulation.

```
student_id = 1;          % Replace with assigned ID, 1 to 20
g = 9.81;
m0 = round(100*(1.31 + 0.02*(student_id-1)))/100;
arm_values = [0.23 0.24 0.25 0.26 0.27];
l = arm_values(mod(3*(student_id-1),5)+1);
m_nom = 1.50*m0;
m = m_nom;
Jdiag = [0.03;0.03;0.05]*(m0/1.5)*(l/0.25)^2;
J = diag(Jdiag);
Omega_max = 800;
kT = 2*m_nom*g/(4*Omega_max^2);
kMT = 0.020;
kM = kMT*kT;
sigma = [1;-1;1;-1];
if student_id <= 10      % Plus configuration
   rotor_xy = [l 0 -l 0; 0 l 0 -l];
else                     % X configuration
   d = l/sqrt(2);
   rotor_xy = [d d -d -d; -d d d -d];
end
c_drag = 0;
X0 = zeros(12,1);
Omega_hover = sqrt(m_nom*g/(4*kT));
Omega_cmd = Omega_hover*ones(4,1);
```

Keep kT, kM, J and Omega_max fixed when applying the specified payload change. These are synthetic coursework parameters. Each column of rotor_xy identifies one rotor in the order used by Omega_cmd.

State indices	Quantity	Units
1–3	NED position pN, pE, pD	m
4–6	Body velocity u, v, w	m/s
7–9	Euler angles phi, theta, psi	rad
10–12	Body angular rates p, q, r	rad/s

Use a 12 × 1 column vector throughout. Position pN must not be confused with roll rate p. Verify the loaded mass, inertia entries, configuration and hover rotor speed against the assigned parameters.

3 Rotor loads and allocation

For rotor i, let its position be [xi, yi, 0] relative to the centre of mass. Rotor speed Omega_i is a nonnegative magnitude. sigma_i specifies the airframe yaw-reaction sign; +1 corresponds to CCW rotor spin viewed from above for these body axes.

$$Ti = kT\,\Omega i^2 \quad Fi = [0, 0, -Ti]^{\mathrm{T}}$$

$$Mi = ri \times Fi + [0, 0, \sigma i\, kM\, \Omega i^2]^{\mathrm{T}}$$

Evaluate the cross product and sum all four contributions. Use the resulting expressions in a MATLAB Function block named RotorLoads. Do not add gravity inside this block.

Interface	Signal and dimensions
Inputs	Omega 4 × 1; rotor_xy 2 × 4; sigma 4 × 1; kT and kM scalars
Outputs	Fprop 3 × 1; Mprop 3 × 1
Units	Omega rad/s; positions m; force N; moment N m

```
function [Fprop,Mprop] = RotorLoads(Omega,rotor_xy,sigma,kT,kM)
% Implement the four rotor thrusts and summed moments here.
Fprop = zeros(3,1);
Mprop = zeros(3,1);
% Replace these initial values with the derived calculations.
end
```

The code above defines the interface only; the calculation must be completed. Connect parameter Constant blocks using the workspace variable names. Confirm fixed input and output dimensions in the MATLAB Function editor if automatic inference fails.

Allocation task

Construct A so that $[\text{Ttotal}, \text{Mx}, \text{My}, \text{Mz}]^{\mathrm{T}} = A\,[\text{T1}, \text{T2}, \text{T3}, \text{T4}]^{\mathrm{T}}$, where Ttotal is the positive sum of rotor thrust magnitudes. Distinguish this from Fprop,z = −Ttotal. Derive each column from the rotor position and reaction sign. Verify rank(A) = 4 in MATLAB.

Calculate the four rotor thrusts at equal hover speeds. Predict Fprop and Mprop and compare with the block output. Explain why zero propulsive moment does not, by itself, imply zero angular velocity.

Geometry checkpoint

Plus numbering: 1 front, 2 right, 3 rear, 4 left. X numbering: 1 front-left, 2 front-right, 3 rear-right, 4 rear-left. Opposite rotors have the same spin; adjacent rotors have opposite spins. For X geometry, d = l/sqrt(2) is the perpendicular lever arm about either horizontal body axis.

4 Frame transformations and dynamics

In a MATLAB Function block named StateDerivative, extract vb = X(4:6), phi = X(7), theta = X(8), psi = X(9), and omega = X(10:12). Form the following matrices using MATLAB sin and cos with angles in radians.

```
Rx = [1 0 0; 0 cos(phi) -sin(phi); 0 sin(phi) cos(phi)];
Ry = [cos(theta) 0 sin(theta); 0 1 0; -sin(theta) 0 cos(theta)];
```

```
Rz = [cos(psi) -sin(psi) 0; sin(psi) cos(psi) 0; 0 0 1];
R = Rz*Ry*Rx;
```

R maps body velocity into NED. Its transpose maps the NED gravity force into body components. Before assembling the dynamics, verify R is the identity at zero angles and check the east-facing velocity example in Section 6.

$$Position\,rate \;=\; R\,vb$$

$$Fg \;=\; R^{\mathrm{T}}\,[0,0,mg]^{\mathrm{T}} \quad Fdrag \;=\; -c_drag\,vb$$

$$Body\;velocity\;derivative \;=\; (Fg \;+\; Fprop \;+\; Fdrag)/m \;-\; \omega \;\times\; vb$$

$$Body\,rate\;derivative \;=\; J^{-1}\,[Mprop \;-\; \omega \;\times\; (J\omega)]$$

Use cross(omega,vb) for the cross product. For rotational dynamics, MATLAB's linear solve J\rhs avoids explicitly calculating the inverse. Gravity is included exactly once in Fg. The baseline includes no aerodynamic moment.

Euler angle rates

```
E = [1 sin(phi)*tan(theta) cos(phi)*tan(theta);
    0 cos(phi)          -sin(phi);
    0 sin(phi)/cos(theta) cos(phi)/cos(theta)];
euler_dot = E*omega;
```

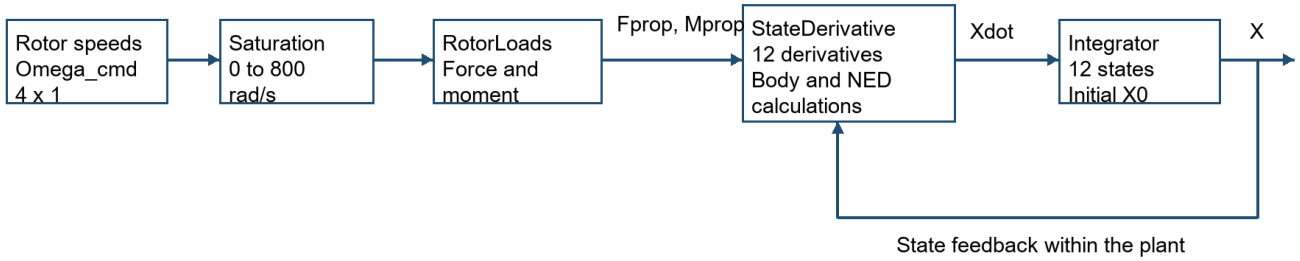
At phi = theta = 0, the angle rates equal p, q and r. This equality does not generally hold at nonzero attitude. Keep pitch away from ±90 degrees; stop a test if the Euler representation approaches its singularity.

Derivative block interface

```
function Xdot = StateDerivative(X,Fprop,Mprop,m,J,g,c_drag)
% Extract states, construct R and E, then implement the equations.
Xdot = zeros(12,1);
% Fill 1:3 with position rates; 4:6 with body velocity derivatives;
% 7:9 with Euler rates; 10:12 with body rate derivatives.
end
```

This is an implementation scaffold, not a completed function. Check the state order and column-vector dimensions before connecting the integrator.

5 Assemble and log the model



Supply parameter Constant blocks to both calculation blocks. Log speeds, loads and states.

Figure 1 Rotor inputs, rigid-body dynamics and the state integrator

1. Use a Constant block with value Omega_cmd and a Saturation block with lower limit 0 and upper limit Omega_max. Its output represents actual rotor speeds for this ideal actuator model. Connect it to RotorLoads.

2. Connect Fprop and Mprop to StateDerivative. Supply m, J, g and c_drag through Constant blocks. Supply geometry and rotor coefficients to RotorLoads in the same way. MATLAB Function blocks do not automatically read arbitrary base-workspace variables; use the explicit parameter ports.

3. Connect Xdot to one continuous Integrator block with Initial condition X0. Feed its output X back to StateDerivative. This feedback implements the state-dependent plant; it is not a flight controller.

4. Add To Workspace blocks for X, applied rotor speeds, Fprop, Mprop and Xdot. Use the variable names X_log, Omega_log, Fprop_log, Mprop_log and Xdot_log. Set each to Timeseries format with sample time 0.01 s.

5. Enable Single simulation output in Model Settings → Data Import/Export and use out as its variable name. Choose variable-step ode45, maximum step 0.01 s, relative tolerance 1e-6 and absolute tolerance 1e-8. Use Stop Time 2 s initially; shorten differential-moment tests to 0.20 s.

Run and save a test

Run the commands below in the MATLAB Command Window after building and saving the model.

```
practical2_parameters
out = sim('Practical2_Quadrotor');
if ~exist('results','dir'), mkdir('results'); end
save('results/hover.mat','out','X0','Omega_cmd','m','J');
ts = out.X_log;
if ts.IsTimeFirst
    X = reshape(ts.Data,numel(ts.Time),[]);
else
    X = reshape(ts.Data,[],numel(ts.Time)).';
end
figure;
plot(ts.Time,X(:,1:3));
xlabel('Time (s)'); ylabel('NED position (m)');
legend('North','East','Down'); grid on;
saveas(gcf,'results/hover_position.png');
```

Use a distinct filename for every test. Run the parameter script to reset between cases, then apply the test overrides before simulating. Do not run the reset script after applying overrides. Save each out object before the next run replaces it.

Before first simulation, update the model diagram to check dimensions. In particular, X and Xdot must both be 12 × 1. Diagnostic force and moment signals must each be 3 × 1.

6 Isolated validation tests

Calculate predictions before simulation. Record the actual numerical outputs separately. Start each test from the nominal script settings unless stated otherwise.

Test	Setup	Required check
Frame mapping	Evaluate R at phi = theta = 0, psi = pi/2. Set vb = [5;0;0].	Verify NED velocity [0;5;0] m/s; check R*R transpose and det(R).
Hover	X0 = zeros(12,1); all speeds Omega_hover; c_drag = 0.	Predict all 12 derivatives and check drift over 2 s.
Free fall	X0 = zeros(12,1); Omega_cmd = zeros(4,1).	Calculate pD and w at 2 s; verify gravity signs and unchanged attitude.
Differential pitch	Start at hover and change thrusts as specified below. Simulate 0.20 s.	Predict initial My and q derivative; check total thrust, roll and yaw moments.
Yaw reaction	Add delta to T1,T3 and subtract delta from T2,T4.	Predict Mz and initial r derivative. Verify roll/pitch moments and total thrust.

Differential thrust commands

Let Ti0 = m_nom*g/4 and delta = 0.05*Ti0. For plus geometry, increase T1 by delta and decrease T3 by delta. For X geometry, increase T1 and T2 by delta and decrease T3 and T4 by delta. In each case convert the target thrust vector to rotor speeds using Omega_cmd = sqrt(T_target/kT). Check all speeds lie between 0 and Omega_max.

Use the appropriate geometry to predict the initial moment and angular acceleration. Initial body rates are zero, so the initial rotational coupling term vanishes. Do not assume later angular rates equal moment divided by inertia; that quantity is angular acceleration, and the full state evolves.

For the yaw test, use the same delta and convert the new thrusts to speeds. The reaction signs must give the yaw direction predicted from sigma. A numerical test of a near-zero yaw moment alone is insufficient to validate yaw signs.

Test	Predicted quantity and value	Simulated value	Difference and explanation
Frame mapping			
Hover			
Free fall			
Differential pitch			
Yaw reaction			

Record initial derivative comparisons separately from final state comparisons. Body velocity derivatives are not generally the inertial acceleration components expressed in body axes; the omega cross vb term accounts for the rotating basis.

7 Payload drag and numerical checks

Payload test

Reset to nominal conditions, then set m = 1.80*m0 while retaining Omega_cmd at the original hover values. Keep J, kT, kM and rotor limits unchanged. With zero initial attitude and rates, predict the initial downward acceleration. Simulate for 2 s and compare pD and w with the constant-acceleration prediction. Explain why the original hover speeds no longer support the loaded aircraft.

Drag test

Reset to nominal conditions. Set X0(4) = 2.00 m/s, c_drag = 0.25 N s/m, and retain level hover rotor speeds. Use still air. Run a second otherwise identical case with c_drag = 0 for comparison.

Predict the initial forward body-velocity derivative from the drag force. With zero attitude and rates, derive u(t) from m du/dt = −c_drag u. Check that forward velocity decays without reversing direction, while the no-drag case retains constant forward velocity. Explain the corresponding North-position traces.

Numerical checks

Repeat one differential-moment test with maximum step 0.005 s, retaining the same initial conditions, inputs and logging interval. Compare final states and logged traces. Report absolute differences with units; do not use percentage error when the expected value is zero. Investigate material changes before adding a controller.

Check rotation orthogonality, determinant +1 and allocation rank four. Check force, moment, inertia and rotor-speed units. Document the assumptions behind each isolated analytical prediction.

8 Required outputs and demonstration

- Parameter script and working model with consistent state ordering and labelled subsystem interfaces.
- Derived allocation matrix for the assigned geometry, including the reaction-torque signs.
- Predicted-versus-simulated test table, saved MAT files and labelled plots.
- Brief explanations of the payload effect, drag response and numerical comparison.

Demonstrate hover and one differential-moment response. Identify where gravity is added and show that the plant input is rotor speed rather than a desired force or moment. Explain one discrepancy or numerical tolerance.